Now that we've previewed the application features, let's discuss the technologies used and their technical considerations.

10.1.2 Technical implementation

The web application uses the key components for building conversational UI web applications that we've discussed in this book, with some additions.

10.1.3 Technology stack overview

Our AI interview assistant is built on a stack of technologies explored in this book, using the strengths of several key tools:

- *Next.js framework*—Next.js provides a versatile environment for building dynamic web applications, supporting both server-side rendering and static site generation.
- *Vercel AI SDK*—Manages LLM and conversational state capabilities, including streaming responses and seamless integration with Next.js server components.

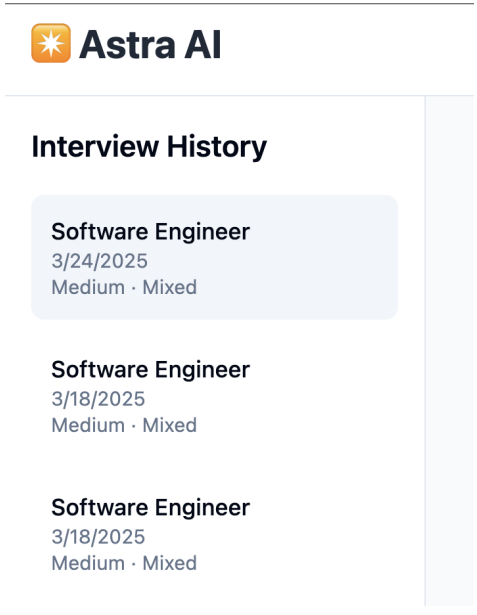


Figure 10.6 The sidebar section will be filled with the current and past interview sessions. Clicking on a session link will load the message history for review.

- *Upstash Redis*—For external data storage, we employ Upstash Redis as a simple yet performant solution. This choice meets our intermediate data needs while allowing for easy scalability to a relational database like PostgreSQL if required.
- *Clerk.js*—For authentication, we utilize Clerk.js, which effectively externalizes the risk associated with managing and handling user authentication and authorization for us.

This combination of technologies enables us to create an initial project that can be further extended with additional features.

Application architecture

The application follows a modular architecture, including a frontend layer consisting of React components with Tailwind CSS for a responsive UI. It uses the shadcn component library for the common components like buttons, alerts, and textboxes.

Now let's look at the individual features and how they are implemented.

ChatThread

This component handles the core chat interface. It displays messages, manages UI interactions, and includes controls for enabling/disabling text-to-speech.